AudioPlatform

for Teensy 4.1

Introduction and Overview

Revision 1.0.1, July 20, 2025

Donald Swearingen and ProtoSupplies.com

What is AudioPlatform?

A Canvas for Sound

AudioPlatform® ***for Teensy 4.1*** is a complete, fully-assembled and self-contained unit supporting the development of a wide range of professional-quality audio systems and devices, from simple to sophisticated.

Based on PJRC's Teensy® 4.1 Development Board and the associated Teensyduino / Arduino tools and libraries, AudioPlatform features a complete inventory of the hardware components necessary for Teensy-based audio projects, as well as additional components providing extended memory and I/O resources.

AudioPlatform consists of two primary elements:

- A compact, adaptable, and portable hardware platform for experimentation and development of stand-alone audio applications.
- A highly-structured and well-documented software application, utilizing the resources of the hardware platform to demonstrate the power and flexibility of audio processing within the Teensy environment.

A Community Effort

Since the introduction in 2014 of the Teensy Audio Library, there have been numerous additions to that code base by a large open-source community of highly-motivated members, whose ideas for engaging audio projects have resulted in a considerable body of impressive and useful software tools. PJRC and the Teensy community have shown that it is not only possible to produce and process high-definition audio, but that the present-day capabilities to do so are at a level on par with audio systems that until recently were far beyond the reach of inexpensive, user-friendly devices. Without this solid base to build upon, AudioPlatform would not be possible.

A Gap Bridged

Despite the potential of Teensy audio, only a small number of complete hardware devices built upon these tools have emerged. And while many of these efforts are quite impressive, they generally are centered around a specific function or task – for example a synthesizer or MIDI controller – with hardware and software unique to the device, rather than a general purpose platfrom that can play host to a variety of audio system designs. AudioPlatform aims to address that deficit, embodying an open and flexible hardware design – usable out-of-the box, no soldering or wiring required – that offers a significant base of resources for audio applications of all sorts.

AudioPlatform hardware and accessories are available at **ProtoSupplies.com.** Documentation and complete source code are on **ProtoSupplies' GitHub Repository.**

AudioPlatform Hardware

The audio-related components of the AudioPlatform hardware baseboard (see ***Bare Baseboard*** below), are derived from the original **PJRC Teensy Audio Shield** design. Additional baseboard components fully extend the storage and IO capacity available for Teensy audio applications.

Hardware Details

- Black Acrylic-Matte Housing and Base
- 800x480 Pixel RGB LCD Capacitive Touch Screen
- Teensy 4.1 @600 MHz with 1 MB RAM
- SGTL5000 Audio Codec
- 8 MB Non-Volatile Program Flash
- 16 MB PSRAM
- 128 MB Non-Volatile Serial Flash Storage
- Teensy 4.1 microSD Card Reader (see note below)
- USB I/O on both MIDI Host and USB-C Connections
- MIDI I/O on both USB and MIDI DIN Ports
- Audio Line In/Out on 3.5 mm Female Connectors
- Audio In/Out over USB
- Headphone Out on 3.5mm Female Connector
- 4 Rotary Encoders for Menu and Device Parameter Settings
- Dimensions: 8.7” Width x 6.2” Height x 2.3” Depth (including encoders)
- Weight: 1 lb 12 oz

Whereas Teensy support of SD cards was initially limited to cards of size 32GB or less, since the release of Teensyduino 1.54 (AudioPlatform currently uses 1.58.0), cards of up to 2TB in size are supported. However, AudioPlatform ships with a 32GB card and has not been tested with cards of larger capacity. Moreover, larger cards may require formatting in a different manner. The subject is discussed in greater detail in AudioPlatform documentation *Vol 3: User Reference Guide.*

AudioPlatform Software Application

The open-source AudioPlatform software application, ***apApp***, is installed and shipped with the AudioPlatform hardware.

apApp Features

- Five Multi-Voice Sound Players: Dexed FM Synthesizer, SD WAV File Player, Serial Flash RAW File Player, Analog-Style Synthesizer, Stereo Audio Input
- SD WAV File Player Plays Stereo WAV Soundfiles Stored on microSD Card, with 8-Voice PolyPhony
- Serial Flash RAW Player Plays Mono RAW (headerless) Soundfiles Stored in 128MB Non-Volatile Flash Storage, with 8-Voice Polyphony
- Analog-Style Synthesizer Plays Sounds Programmed By Users, with 12-Voice Polyphony
- FM Synthesizer Features 512 Voice Presets in 16 Banks of 32 Voices, Including All Voice Banks of the Original DX7, On Which Dexed Is Based; 10 Banks Stored in Non-Volatile Program Memory; 6 User Banks Loadable from SD Card at Startup
- Stereo Reverb and Stereo Ping-Pong Delay
- All Players and Effects Simultaneously Active or Inactive Based on Preset Settings (no Mutually-Exclusive Devices)
- Player and Effects Parameters Stored in Recallable Preset Banks, 32 Presets per Bank, Unlimited Bank Storage on microSD Card with Instant Recall
- Flexible System Architecture with Individual Components (Players/Effects, Other Elements) Modifiable and/or Replaceable
- Extensive Display and System Support with C++ Libraries, Many of Which Are Application-Independent
- TouchScreen and Encoder Support for Screen Selection, Editing and Selection of Player, Effects, and System Parameters
- Real-Time Clock Support
- Real-Time Display of Audio Levels, CPU and Memory Usage
- User Utilities for File Viewing and Management

Of particular note is the fact that use of the *apApp* software application is not required in user designs. Developers who wish to adapt the hardware elements – and, optionally, elements of the open-source software – in their own designs and projects will find AudioPlatform's resources to be fully supportive of their efforts, freeing them to proceed without the need to acquire or construct additional circuit components.

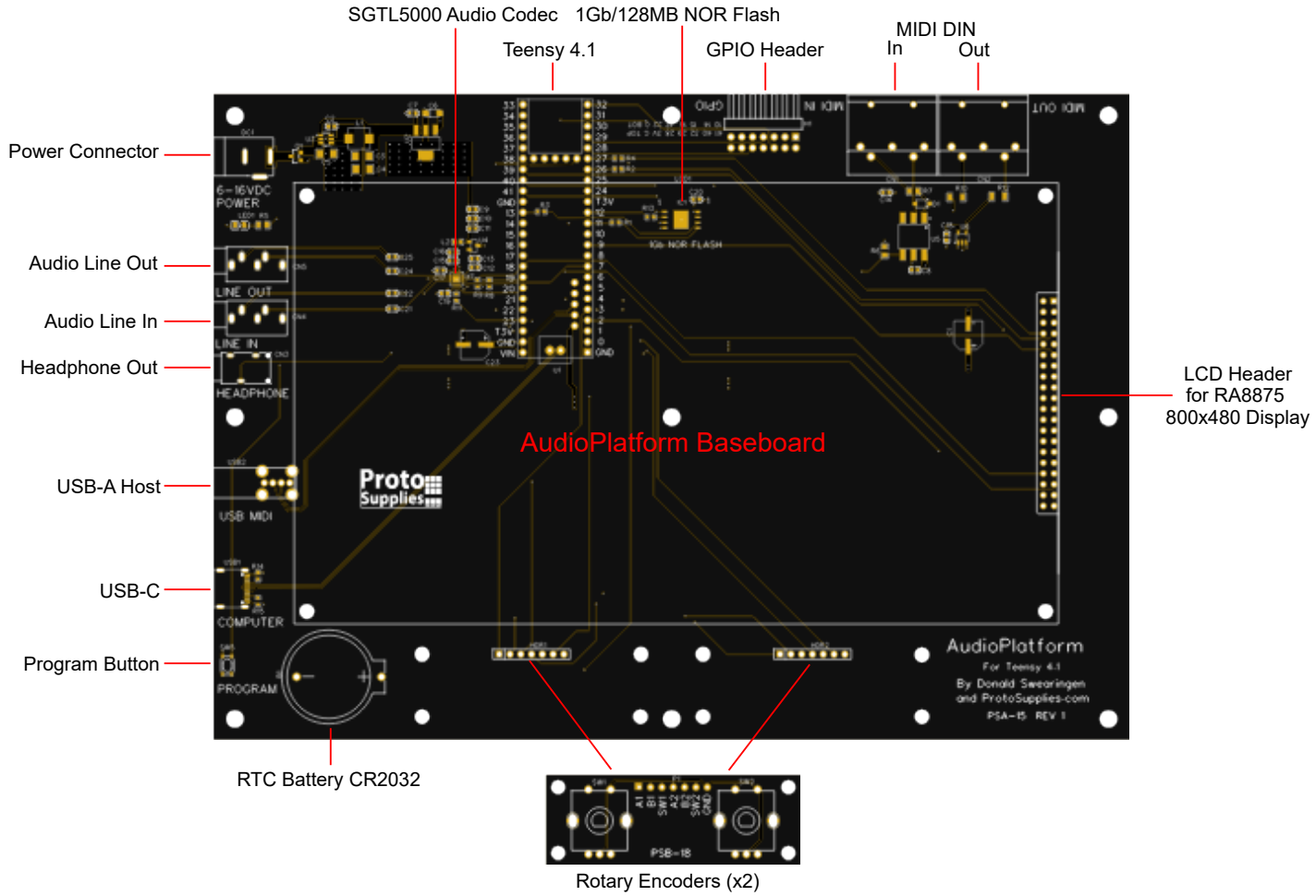
Hardware Overview

AudioPlatform PCB

The AudioPlatform PCB baseboard design includes all the components required by applications based on the Teensy Audio Library and the Teensy 4.x Audio Shield, as well as additional components that provide for physical IO and expanded memory capacity.

Bare Baseboard

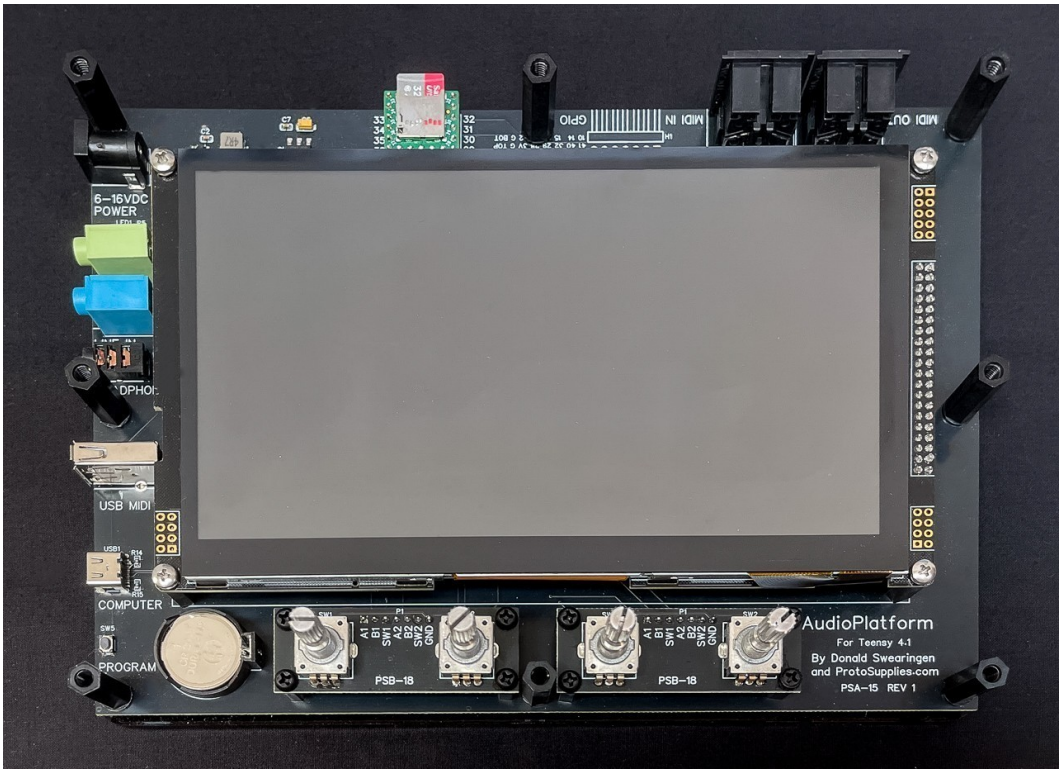
The photo image below shows the bare PCB board that is mounted between the top and bottom acrylic plates, with the LCD display removed. All of the IO ports are accessible on sides and rear of the assembled unit through standard connectors for power, audio, MIDI and USB.



In addition to the the IO and power connections, the PCB design incorporates two design features that facilitate user access: the Teensy 4.1 is positioned such that micro-USB card can be readily be inserted and removed at the rear of the unit; and the standard Teensy micro-USB programming connector has been extended to the PCB edge as a USB-C connection (the older micro-USB connectors are often more difficult to connect to, and have a tendency to break off from their surface-mount soldering if too much force is applied).

Baseboard With Components

The image below shows the PCB baseboard fully-populated with its components in place, and ready for the addition of its matte-acrylic top panel.



Note that the RTC battery is conveniently located such that the CR2032 battery can be replaced without the need to remove the top panel. The battery can be released by pressing the clip on the right side with a finger or small tool, releasing the spring-loaded battery, which pops up and can then be easily removed.

Input / Output Connections

Power 6-16V DC

Power is delivered through the DC input connector from a standard DC wall adapter with a center-positive (+), 2.1mm female connector. Note that power normally supplied by a computer when connected to the AudioPlatform USB-C connection is disabled – AudioPlatform can only be powered from the DC input.

Audio Line In / Line Out

Stereo line-level inputs and outputs are female 3.5mm TRS connectors.

Headphone Out

The headphone output is a 3.5mm TRS female connector.USB-A Connector

USB-A Connector

The USB-A connector (labeled “USB MIDI” on the PCB) is used by the AudioPlatform application for MIDI input from a MIDI keyboard or controller, but can be used in any standard manner by other applications. The connection can also supply power to a connected device, up to a maximum of 500mA—higher current devices must be externally powered.

USB-C Connector

The USB-C connection is used exclusively for connection of AudioPlatform to a computer and supports both the Arduino IDE and Serial IO, in addition to many other assignable options.

Program Button

The small program button is employed in the process of downloading a per-complied executable (“.exe”) image to the AudioPlatform’s Teensy 4.1, using the *Teensy Loader* application.

microSD Card Slot

The microSD (µSD) slot accepts a standard microSD card. Cards with "A1", "A2" or higher ratings are recommended for best performance.

MIDI DIN In/Out

MIDI input and output is also possible on the 5-pin DIN connector (original MIDI standard still used with numerous current MIDI devices). MIDI input and output uses Teensy’s RX1 and TX1 pins.

Rotary Encoders

The Rotary Encoder Module, a small independent PCB, provides 2 continuously variable position sensors that report the relative position of the shaft, the direction of rotation and whether or not the shaft is depressed (on/off switch). AudioPlatform uses two encoder modules that attach to the baseboard using the two 1x7 pin female headers at the bottom of the AudioPlatform PCB.

LCD Header

The LCD header block is the plug-in slot for AudioPlatform’s 7” RA8875 capacitive touchscreen display.

RTC Battery Socket

A socket for a 3V CR2032 battery is located on the lower-left of the AudioPlatform PCB, accessible without disassembly or removal of the top panel.

Program Button

The small button on the lower left of the PCB is used to initiate download of pre-compiled EXE files using the stand-alone Teensy Loader application.

PCB Components

Processor

Teensy 4.1 ***Hybrid Circuit Device*** is based on one of the most powerful microcontroller devices (MCUs) currently available, the NXP IMXRT1062, with the capacity to process complex, high-definition audio applications in real-time.

The IMX RT1062 MCU is a 32-bit ARM 7 device running nominally at 600 MHz. The processor can be overclocked by selecting various clock rates (720MHz, 816MHz, and higher) in the Arduino IDE when building applications.

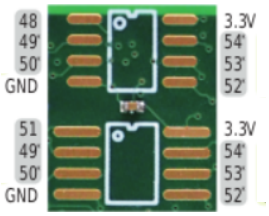
AudioPlatform has been successfully tested at the 720MHz and 816MHz rates, but sticks with the basic 600MHz rate since it is sufficient for the current requirements of the AudioPlatform application. At even higher clock rates, the concern is that at some point over-heating may become an issue without additional cooling components.

Program Flash (PROGMEM)

As its name indicates, the program flash features 8MB of storage, populated at compile and load time with the compiled EXE image. Sections of PROGMEM can also be reserved for use as a file system accessible by the Teensyduino LittleFS library (the AudioPlatform application reserves 1MB for non-volatile storage of persistent objects), and can store components of software applications (arrays, bitmaps, fonts, etc) which are declared with the PROGMEM keyword.

PSRAM (EXTMEM)

Program objects declared with the EXTMEM keyword are referenced in the PSRAM space. AudioPlatform is configured with two 8MB PSRAM chips (AP 6404L-3SQR) soldered to pads on the underside of the Teensy 4.1 Development Board:



The 2 chips are serial QSPI (Quad SPI) devices, addressable as conventional memory, that transfer data on 4 data parallel serial lines. That is, even though the device is a parallel-serial device, details of data access are handled within the MCU's architecture such that the PSRAM can be addressed as though it were a normal contiguous memory space.

Serial NOR Flash

Teensy Teensy 4.1 development boards can utilize serial flash memory chips for expanded storage capabilities. AudioPlatform includes a WinBond W25Q01JV 1Gb (128 MB) SPI NOR via a SPI serial interface (SPI). The device can be used for storing and retrieving data using the file system implemented by the Teensyduino LittleFS library.

While SPI flash access is of course much slower than that of internal RAM or PSRAM, it is nevertheless more than fast enough to play audio files stored in RAW audio format, and far faster and more efficient in accessing and playing audio files than the microSD card, and is thus used primarily in this capacity by AudioPlatform. The 128 MB storage capacity is capable of storing up 20 to 25 minutes of 44.1 kHz 16-bit RAW audio data.

microSD Card

The Teensy 4.1 development board includes a microSD slot, accessible on AudioPlatform via a cutout in the top panel. The SD card is used primarily for storing stereo WAV audio files, but also is used to store preset banks and FM synthesizer banks.

Since version 1.54, the Teensyduino library supports cards larger than 32GB (when formatting, use FAT for 32GB and smaller cards, or exFat for larger than 32GB). However, AudioPlatform ships with a 32GB card, and larger cards have not been tested as of the initial release. It is recommended that cards of 32GB or less be used with AudioPlatform.

USB I/O

Various types of USB input and output are supported by the Teensyduino environment and Arduino IDE.

LCD TouchScreen

The RA8875 TFT LCD controller features 800x480 RGB pixel resolution. It features a built-in 768KB display RAM, enabling buffering and even double buffering for smoother display output. The RA8875 is addressed as a 4-wire SPI device using 4 pins on the Teensy 4.1 (see ***Teensy 4.1 Pin Assignments*** below). It also incorporates a hardware font engine, 2D block transfer engine (BTE), and geometric shape acceleration, making it suitable for both text and graphics applications.

RTC Battery

The Teensy 4.1 IC has a built in real-time-clock (RTC) module, powered by a CR2032 3V coin-cell battery inserted in the battery socket. The time is automatically set to the current time of a connected computer whenever an executable (EXE) file is downloaded to the device, either from the Arduino IDE, or when using the independent Teensy.exe loader. When using the AudioPlatform *apApp* software application, the time can be set by the user via the *Set Time* screen, accessed through the Utility Menu.

Teensy 4.1 Pin Assignments

The following diagrams show the assignments of all the Teensy 4.1 pins, including both those used by AudioPlatform, and those available for use via the GPIO expansion port.

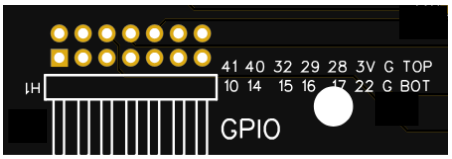
Complete Pin Usage Overview

AudioPlatform for Teensy 4.1 Pin Assignments

Color Key		PWM	Digital Audio	Serial	Digital		Analog	I2C	SPI	CAN Bus	Power		
Baseboard													**Baseboard**
Power	GND				GND		VIN					VIN	5V Power
MIDI IN	RX1	PWM	CRX2	CS1	RX1	0	GND					GND	Ground
MIDI OUT	TX1	PWM	CTX2	MISO1	TX1	1	3.3V						No Connection
RA8875 Touch	INT	PWM	OUT2			2	23	A9	CRX1	MCLK1	PWM	MCLK	SGTL 5000 Audio
Rotary Encoder 1	CLK_A	PWM	LRCLK2			3	22	A8	CTX1		PWM		I/O Pin 11
Rotary Encoder 1	DT_B	PWM	BCLK2			4	21	A7	RX5	BCLK1		BCLK	SGTL 5000 Audio
RA8875 TFT Display	CS	PWM	IN2			5	20	A6	TX5	LRCLK1		LRCLK	SGTL 5000 Audio
Serial Flash CS	CS	PWM	OUT1D			6	19	A5		SCL	PWM	SCL	SGTL 5000 Audio / RA8875 Touch
SGTL5000 Audio	DIN	PWM	OUT1A	RX2		7	18	A4		SDA	PWM	SDA	SGTL 5000 Audio / RA8875 Touch
SGTL5000 Audio	DOUT	PWM	IN1	TX2		8	17	A3	TX4	SDA1			I/O Pin 9
RA8875 TFT Display	RESET	PWM	OUT1C			9	16	A2	RX4	SCL1			I/O Pin 7
I/O Pin 1		PWM	MQSR	CS		10	15	A1	RX3	SPDIF IN	PWM		I/O Pin 5
Serial Flash MOSI	MOSI	PWM	CTX1	MOSI		11	14	A0	TX3	SPDIF OUT	PWM		I/O Pin 3
Serial Flash MISO	MISO	PWM	MQSL	MISO		12	13	LED	SCK		PWM	SCK	Serial Flash SCK
No Connection						3.3V	GND					GND	Ground
Rotary Encoder 1	SWITCH	PWM	SCL2	TX6	A10	24	41	A17					I/O Pin 2
Rotary Encoder 2	CLK_A	PWM	SDA2	RX6	A11	25	40	A16					I/O Pin 4
RA8875 TFT Display	MOSI1			MOSI1	A12	26	39	A15	MISO1	OUT1A		MISO1	RA8875 TFT Display
RA8875 TFT Display	SCK1			SCK1	A13	27	38	A14	CS1	IN1		SWITCH	Rotary Encoder 4
I/O Pin 10		PWM		RX7		28	37		CS		PWM	DT_B	Rotary Encoder 4
I/O Pin 8		PWM		TX7		29	36		CS		PWM	CLK_A	Rotary Encoder 4
Rotary Encoder 2	DT_B		CRX3			30	35		TX8			SWITCH	Rotary Encoder 3
Rotary Encoder 2	SWITCH		CTX3			31	34		RX8			DT_B	Rotary Encoder 3
I/O Pin 6			OUT1B			32	33			MCLK2	PWM	CLK_A	Rotary Encoder 3

Teensy 4.1

GPIO Expansion Port Pins



The GPIO port is an unpopulated 7 pin x 2 row through-hole socket suitable for a 7x2 right-angled male header intended for connection of external circuits and devices to Teensy 4.1 pins not used by AudioPlatform, as shown in the diagram below:

				PWM	PWM			
	A17	A16	OUT1B	TX7	RX7			**Teensy Pins**
	41	40	32	29	28	3.3V	Gnd	
Top	2	4	6	8	10	12	14	**GPIO Pins**
Bottom	1	3	5	7	9	11	13	
	10	14	15	16	17	22	Gnd	
	CS	A0	A1	A2	A3	A8		
	MQSR	TX3	RX3	RX4	TX4	CTX1		**Teensy Pins**
	PWM	SPDIF OUT	SPDIF IN	SCL1	SDA1	PWM		
		PWM	PWM					

Color Key
Digital
PWM
Serial
Digital Audio
Analog
I2C
SPI
CAN Bus
Power

Software Overview

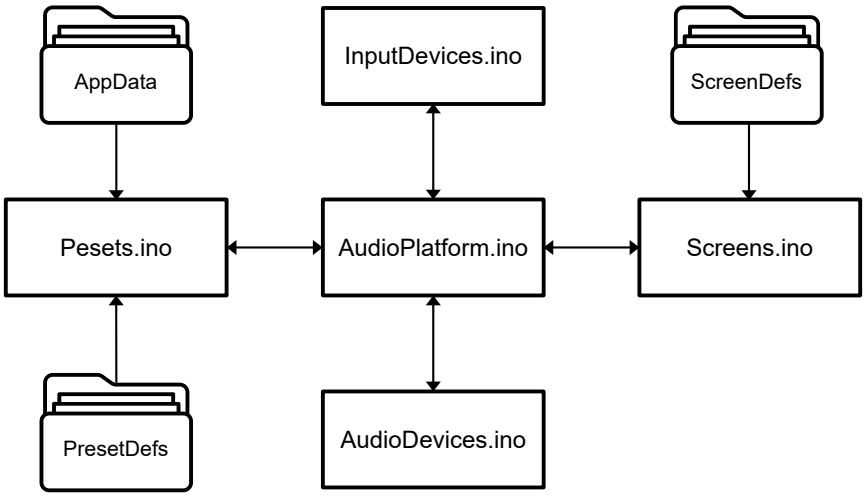
This section provides a basic overview of the *apApp* application, its components, and organization. Detailed coverage, including instructions on how to navigate, edit and build the application, are provided in the document *Vol.4 Application Reference*.

Application Architecture

Components of the *apApp* software are organized in two main folders: *Projects,* containing the core components of the application, and *Libraries,* which includes a number of sub-folders representing the various C++ classes, both application-specific and application-independent, required by the application.

Projects Folder: Core Components

The *Projects* folder includes the 5 *core components* of the application, and several sub-folders, the contents of which are required by the application. All of the core components are written as standard C-language modules with the standard Arduino extension ".ino". In general, the core modules access functions and data structures in other core modules using *namespace* tags (for example, which serve both to emulate that of C++ static classes, and to clearly indicate the location of the item being accessed).



Application Core Components

Customarily, in mixed C and C++ projects, the statement *extern "C"* is generally required to ensure compatibility when linking C++ code with code written in C. In the context of the Arduino ecosystem, where the code is inherently C++-based, using extern "C" is not automatically required because the environment implicitly handles C++ compatibility.

AudioPlatform.ino, AudioPlatform.h

AudioPlatform.ino is the “main” program of the AudioPlatform software: the location of both the standard Arduino *setup()* and *loop()* functions. *setup()* performs various initialization activities in a specific order designed to ensure that objects and functions required in the various core application components are defined and available as each module’s initialization is performed. It’s first action is to query the hardware and populate a global system profile structure (*SystemHardwareProfile*) to be shared among core AudioPlatform components, as well as indirect access by selected screens and support classes when passed as a constructor argument.

AudioPlatform.h defines numerous data types, terms and structures used in the various core components, as well as include statements for their individual .h files, making their exported namespace functions available to the main program.

The standard Arduino *loop()* function is called repeatedly, without delays, to poll individual core components, by which process the components are able to carry out their various activities at a rate compatible with the real-time nature of the system.

AudioDevices.h, AudioDevices.ino

The AudioDevices component defines the structural elements (variables, types, methods, and terms) of the audio components of AudioPlatform, based on classes and objects defined in the Teensy Audio Library as well as classes and objects defined in the *Libraries* folder.

At startup, AudioDevices instantiates, initializes, and connects the inputs and outputs of all audio devices and components used in the application.

After initialization in the AudioiDevices component, real-time audio processing occurs at the interrupt / DMA levels under control of the Teensyduino Audio library, and is thus independent of the high level polling conducted in the *loop()* function.

InputDevices.h, InputDevices.ino

The InputDevices component is responsible for the initialization, runtime polling, and input handling of the various AudioPlatform input devices, including encoders, buttons, MIDI controller, and the MCU’s realtime clock (RTC).

Presets.h, Presets.ino

The Preset component might more accurately be described as the application’s data manager (in fact, plans for future application revisions include the renaming of the component to *DataMgr).* At runtime, the Preset component manages access and storage of presets and preset banks, as well as the communication of preset data and data modifications between the various other core components including both device and user inputs.

Runtime user interactions with encoders and the touchscreen are passed to the Presets component via callback functions passed as arguments to the constructors of various display screens (for example Presets::setBasicSynthParam() when the BasicSynth screen is active). The callback

functions, in turn, act upon the data as appropriate, storing changes to preset settings, and communicating the changes as appropriate to various other system elements . For example, a change to the Basic Synth screen will be stored, and then sent to the AudioDevices component function AudioDevices::setBasicSynthParam() for further action.

The Preset component also is solely responsible for storage and retrieval of user preset and other system data to and from the Teensy 4.1 microSD card.

PresetDefs Folder

The *PresetDefs* folder contains 2 files used primarily by the Preset component

- *PresetDataStructs.h:* defines the data structures and settings of presets the Player and Audio devices of the application, and subject to storage and recall from preset banks.
- *PresetDefaultData.h:* establishes default values used to first initialize the data structures of new presets, based on the structures defined in PresetDataStructs.h.

AppData Folder

The *appData* folder is devoted to terms and structure definitions for various audio and system devices, especially when the device requires access to large data sets that require non-volatile storage. In the Teensy 4.1 device, this storage is provided by the 8MB external Flash bank, of which the application requires less than 1MB, leaving the rest available for storage of varous items that require non-volatile storage.

In the initial release of the AudioPlatform application, the *appData* folder contains only 2 files, both of which are related to the application's FmSynth device, a Teensy-based version of the popular Dexed emulator, which itself emulates the classic DX7 synthesizer.

- *FmSynthPresetBanks.h:* hex/byte array definitions for the various FM Synth voice banks used by the Dexed DX7 emulator and stored in non-volatile PROGMEM.
- *FmSynthPresetConst.h:* definitions and constants related to the storage and management of FM Synth voice banks and presets by the AudioPlatform software application

Screens.h, Screens.ino

AudioPlatform display screens are largely data-driven instances of a collection of display classes defined in various *Library* folders. the appearance and behavior of which are determined by the data structures and elements defined here.

At any given moment, only one screen is active and on display. Every screen represents an instance of one of several *base classes*, some of which are application-specific, and others reusable generic screen definitions. Ultimately, however, all screen instances ultimately are based on the single static class *DisplayScreenBase* in the library *Generic_DisplayClasses.*

DisplayScreenBase defines a number of static elements common to all screen classes – its static methods and structures are shared among all derived screens through the static *tftDisplay* object which provides a layer of indirection between screens, display types, and elements defined in

application subclasses, allowing the physical display (RA8875, ILI9341, etc.) to be changed without modifications to existing screen code.

Additionally, non-static DisplayScreenBase objects and functions provide a means for sub-classes to implement their own definitions specific to their intent and requirements, for example *updateScreen()*.

ScreenDefs Folder

The ScreenDefs folder includes a .h definition for every single screen used in the AudioPlatform application (for example *MainMenuScreenDefs.h*). Screen definition files generally include structures and definitions that define the layout and content of the various display screens. Each screen definition file concludes with a statement creating an instance of the screen being defined, with appropriate constructor arguments,

The entries in ScreenDefs folder are referenced exclusively and only by Screens.ino, making it the owner of all application screen instances.

Libraries Folder: Support Components

The design of the application is such that the majority of the *Library* classes are application-independent, with no awareness or access to the definitions, code, and structures defined in the *Projects* folder, and are therefore usable in contexts other than AudioPlatform. The exception to this is that a number of application library classes are able to access certain core structures and functions indirectly via references supplied to a class in its constructor arguments.

AudioEffects

Audio effects classes: Plate Reverb and Stereo Ping-Pong Delay, Granulator in development.

AudioObjects

Custom audio classes: 8 channel mixer, Audio In/Out, Pan, Gain, Envelope.

AudioPlatform_ScreenClasses

Individual, application-dependent classes for all application screens and derived classes.

AudioPlatform_SupportClasses

Global application-dependent definitions, Serial Flash support, BitMapData, etc.

BasicSynth

Application-independent code for Basic Synthesizer player based on Teensy Audio library.

EncoderTool-master

Open-source encoder tool from **https://github.com/luni64/EncoderTool**, installed as library in Arduino IDE as supplement to Arduino/Teensyduino encoder handling.

FmSynth

Multi-voice wrapper and extension class for MicroDexed DX7 emulator.

Generic_DisplayClasses

Application-independent, hardware-independent display classes: alphanumeric keyboard, bitmap display, text editing, list selection, etc.

ILI9341_fonts-master

Generic fonts (usable on multiple LCD displays) as hex data, included as replacement/additions to the fonts in Teensyduino's standard ILI9341 support library, which is removed from the Teensyduino library because it is not required by AudioPlatform.

Downloaded from **https://github.com/PaulStoffregen/ILI9341_fonts/tree/master** and placed in user *Libraries* folder.

MediaPlayers

Custom player classes for playback of audio files from the microSD card and Serial Flash.

https://github.com/FrankBoesing/Teensy-WavePlayer

play_wav.cpp, play_wav.h: Advantages and problems

MidiSupport

MIDI support classes: MIDI IO, Voice Manager, MIDI and MIDI controller definitions.

Synth_Dexed

Synth_Dexed is a port of the Dexed sound engine (**https://github.com/asb2m10/dexed**) for Teensy. Synth_Dexed (**https://codeberg.org/dcoredump/MicroDexed**) is compatible with all SYSEX files created for the lenegdary DX7 synthesizer.

Teensy40_Util

Various utilities compatible with Teensy 4.0 devices.

Teensy41_Util

Various utilities compatible with Teensy 4.1 devices.

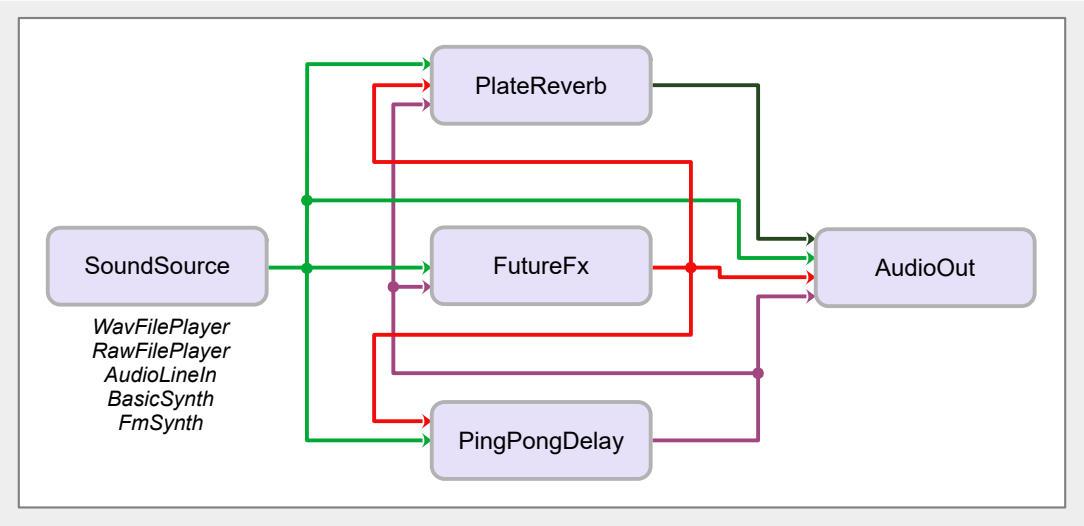
Utility

Generic application-independent C++ utility classes

Audio Architecture

Basic Signal Flow

The general design of AudioPlatform encompasses 5 sound sources (4 sound *players* + audio line in) and 3 effects devices (one of which is unpopulated in the initial software release, and reserved for future development), for a total of 8 signal paths, which are conveniently handled by a number of 8-input mixers (based on the Teensy Audio library). The overall design is shown here, followed by a detailed drawing of the numerous connections of outputs to inputs.

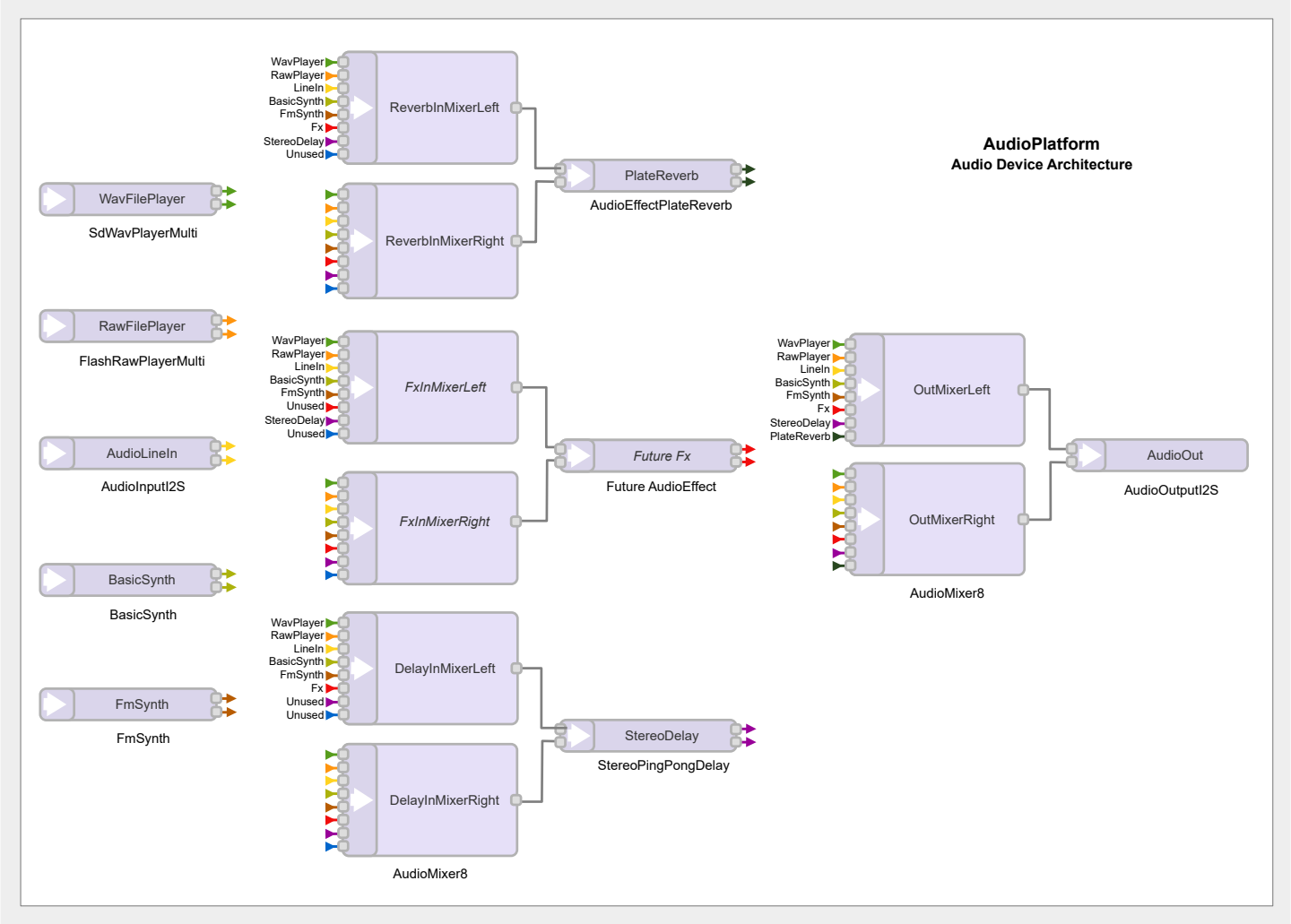


The flow of signals in the diagrams is basically left-to-right, sound sources to effects devices to outputs, with signals routed back from the outputs of some effects devices to inputs of other effects devices.

Connection Detail

The overall signal connection scheme of the *apApp* is depicted in the illustration below (connections are denoted by color rather than by drawing lines – as in the previous diagram – which would make the diagram below both difficult to create, and even more difficult to follow).

Note that the entire scheme is built around *stereo* signal paths, as indicated by the dual arrows of the same color shown as outputs from some devices, and including the need for 8-channel mixers for each of the left and right channel signal paths.



The design of Teensy Audio library restricts device and effects inputs to a single signal line. Device and effects outputs, however, can be connected to multiple inputs, as is shown in the color coding of the connection diagram. Though it may appear that the double output lines of some of the devices in the diagram seem to violate this restriction, they actually represent 2 separate outputs for the left and right channels.

Development Environment

A few details of the *apApp* software development environment are discussed below in overview. Specific and extensive details of the application design, components, and operations are covered in the documentation's *Volume 4: Application Programming Reference*

Arduino IDE

apApp is designed to be compiled within the Arduino IDE. There are numerous other means by which experienced programmers can build the application, perhaps even abandoning the *setup() / loop()* scheme of the Arduino model entirely. However, for the sake of familiarity and the vast extent of usage and user experience with the standard Arduino model, *apApp* assumes that users who wish to modify or extend the application (or simply to build with different options enabled) will compile and download the application in the Arduino IDE.

The two *Projects* and *Libraries* folders are thus assumed (and required) to be located in the *SketchBook* folder of the Arduino IDE (configured by the user in the Preferences section of the Arduino IDE application).

As explained in the *Application Reference* , in order to avoid potential conflicts with existing user libraries and sketches, it is strongly recommended that only the two main folders described below, *Projects* and *Libraries* be present in the IDE's specified Sketchbook folder when first working with and building the application.

Teensyduino and 3rd Party Extensions

apApp is built upon the Teensyduino environment and requires that it be installed in order for the application to be successfully compiled by the Arduino IDE. In addition, there are a number of third-party libraries that are required by the application. A complete accounting of the required extensions is provided in the *Application Programming Reference*.

Development Tools and IDEs

In order to fully and conveniently navigate and become familiar with the AudioPlatform application, users will find that the editor in the Arduino IDE is entirely insufficient, bordering on useless. Instead, the code base must be edited in a more powerful environment designed for management of extensive software environments.

apApp is edited and managed in an Eclipse IDE environment (a well and firmly established environment in embedded systems development). Others have successfully setup large Arduino applications in IDEs such as VS Code, PlatformIO, and Visual Studio. All of these are all software development tools, specifically integrated development environments (IDEs) and code editors, used for writing, debugging, and managing code. Eclipse and Visual Studio are more comprehensive IDEs, while VS Code is a lightweight code editor with strong extension support, and PlatformIO is a development environment focused on embedded systems and IoT.

Creation Story

Motivation (A Personal Note)

Inter-Disciplinary

I'm a life-long professional musician and composer. During my undergraduate music studies at university, I was encouraged to enter a new program of inter-disciplinary studies, which allowed me to expand my focus into the domains of mathematics, physics, electronics and computer science, with their intimate relationship to the domains of sound and music.

By the time I entered graduate school in mathematics, with a focus on the mathematical foundations of computer science, I was fully committed to a future that would integrate all of my fields of interest in my artistic efforts, with computers at the focus of my explorations.

Beginning by opening up early modular synthesizers to examine their parts, and learning some basic workbench skills, over time I acquired considerable skills in software and electronic systems development, working as a self-employed contractor in the San Francisco Bay Area, primarily in embedded systems development. And I've been able to utilize these same skills to design and build numerous sensor-based software and hardware devices and systems in support of music and artistic projects, both for myself and for other performing artists.

Too Much Gear

Over the past 30 years or so, we've seen the emergence of powerful desktop and laptop computers that gradually began to replace the large number of devices one was once required to lug around for performances – a 1996 tour in Japan involved several 4-space rack-mount equipment cases, other cases for cables and assorted items, a full-size sampling keyboard, a MIDI-only laptop computer, sensors, and other gear that today would be virtually impossible to carry along on a flight these days and only at great expense.

Less Gear, New Headaches

Much of this large volume of audio gear has today been replaced by my Mac laptop and a collection of software applications, but there is still a good amount of equipment (support table, keyboard, audio interfaces, sensor devices, mixers, etc.) to be carried around, setup and dismantled. But this new arrangement has brought with it a whole new set of issues that must be repeatedly navigated, as computer hardware and applications are constantly updated, often rendering working hardware, software and other elements broken and unusable.

Like so many of us, I shudder to consider the hundreds of hours I've spent over the years simply finding fixes or work-arounds for these sorts of issues – a situation familiar not only to professionals, but to literally eveyone, given the proliferation of smart phones and other digital devices and software (ever try to get a human at Adobe or Xfinity?).

A Simpler Approach?

In this milieu, I'd often look over from my work desk at my 20+ year-old Yamaha Motif7 keyboard and would think: "when I turn that thing on, it always does *exactly* the same thing it did the day I brought it into my studio." But most everything else in my studio was inextricably locked into the corporate ecosphere with its expensive updates, and constantly moving horizons.

I began to wonder if it might be possible to recreate that "works-every-time" experience, if only at a small scale, for use in selected, more informal settings. Wouldn't it be nice if I could now and then simply carry a lightweight MIDI controller and a single, small audio device that would neverless suffice to create an engaging and impressive presentation? This became the seed of a project I hoped would eventually mature and become just such a device.

Limited Options

In the mid-1990s, there weren't many options for designing and ordering PCB boards in small quantities. But over the ensuing years, things improved steadily to the point where one could design PCB layouts using free software, and then order a small quantity at low cost.

But as components became smaller and more powerful, PCB designs became more complex and incresingly dominated by surface-mount-devices (SMDs), which were beyond the scope of my skills. In addition, my personal projects invariably relied on the use of desktop computers and software to communicate with sensor devices to generate and control audio video outputs in real-time.

Meanwhile, in my embedded systems work, I was exposed to the advancing power of various 32-bit microcontrollers (MCUs), eventually being engaged to design software for two projects that employed LCD video touchscreens with update rates of 60 frames/second, based on two separate MCUs: NXP's LPCXpresso54628, and STM's STM32CubeH7. On exposure to the speed and power of these devices, I began to wonder what sort of support such processors could provide for real-time generation and processing of audio data.

Enter Teensy

In the early 2000s, on a friend's recommendation, I began using Teensy devices in many of my sensor designs. Teensy processors were not only powerful, but had the considerable advantage of a huge library of support software (*Teensyduino*) for all sorts of devices and components. I eventually became aware that Teensy was increasingly being used in audio projects, with an extensive open-source audio library.

Supply-chain issues of the early 2020s reduced the available Teensy devices to the two models based on the NXP IMXRT1062, the Teensy 4.0 and 4.1 However, this was fortuitous for audio enthusiasts—the Teensy 4 series devices being exceptionally capable of real-tme audio processing. This is the point at which I decided to jump in the water and explore what sort of audio systems and designs might be possible with this device.

Donald Swearingen
August 2025

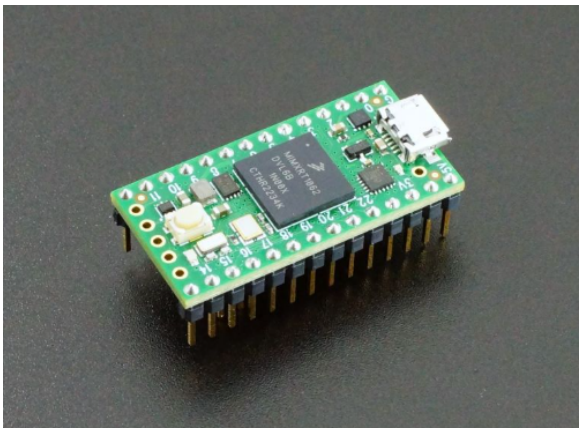
Teensy Audio Design

Beginnings

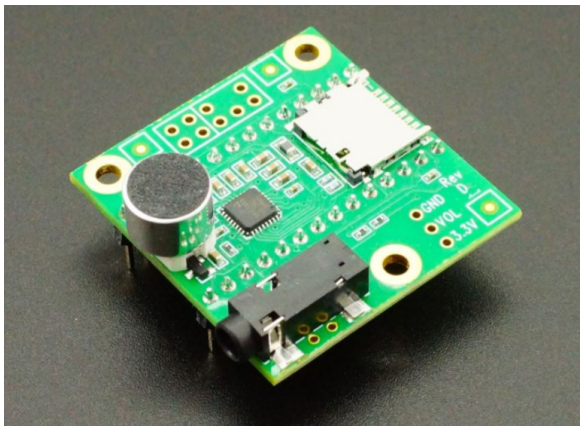
Since its introduction, the **Teensy Audio Shield** has been the subject of countless experiments and explorations of audio in the Teensy community; and the SGTL5000 audio codec (small chip just left of center in the image below) used on the Audio Shield has remained a staple of Teensy audio systems, including AudioPlatform.

Audio Stack

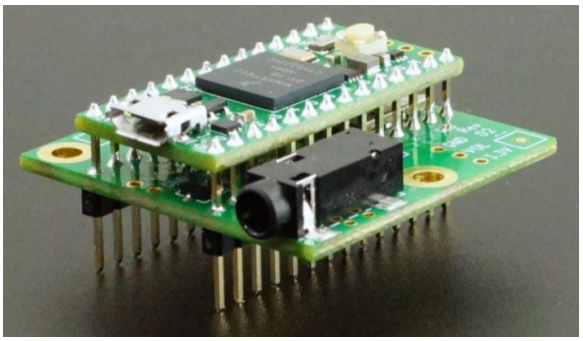
In the initial release – and still in many current situations – the Audio Shield was expected to be mounted beneath and soldered to a Teensy 4.0 device, whose relavent pins were thereby connected to the shield, to create an “Audio Stack”, as shown in the following images.



Teensy 4.0



Teensy Audio Shield



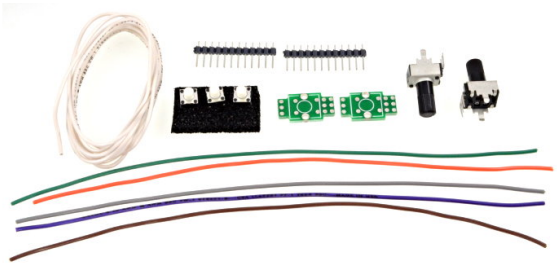
Teensy Audio Stack

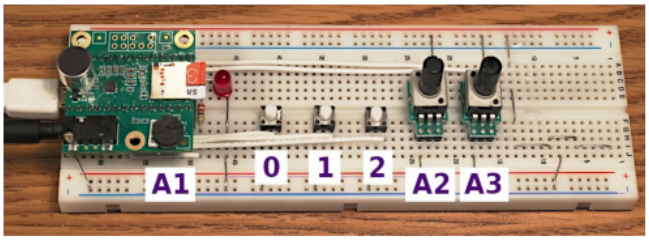
Once the two devices were thus coupled, they could then be inserted into a “breadboard”, which could be used to perform various experiments outlined in the extensive step-by-step **Teensy Audio Tutorial**, and an informative video on the Teensy web site.

Tutorial Kit and Tutorial

On its introduction, the audio shield was supplemented by a tutorial kit, separately purchased, consisting of a few components (image below) that could be added to a breadboard, along with the Teensy audio stack, to create a setup which could be used for carrying out all the experiments and lessons in the tutorial.

The design relies on two key features: Provision of power to the Teensy and Audio Shield via the standard USB micro connector on all Teensy devices used for programming and communication with the Teensy; and audio output from the shield via a headphone connector on the shield.



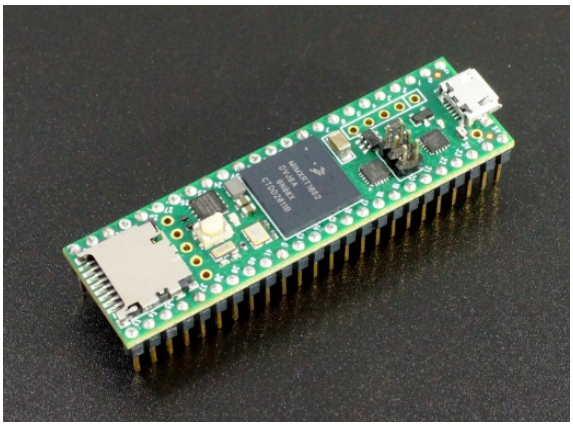


Tutorial Kit and BreadBoard

In the image above, the Audio Shield is mounted on top of the Teensy 4.0, as opposed to the image on the previous page, which show the shield mounted beneath the Teensy 4.0. Both orientations work, but topside mounting does not require removal of the shield's microphone (small cylinder on the left), which is required in several of the tutorials.

Teensy 4.1

Even more resources for systems designs are provided by Teensy 4.1, which uses the same MCU as Teensy 4.0, but offers access to a larger number MCU pins that provide additional features and IO options such as support for Ethernet, QSPI PSRAM and Flash Memory, and additional Program memory, among other features. So this was the device I chose to work with in explorations of Teensy audio.



Teensy 4.1 Hybrid Integrated Circuit

Breadboards and Jumpers

The image of the breadboard assembly used for the tutorial pretty much exemplifies the setup that has been employed, with some variations, by numerous users, both novice and veteran, for exploring and testing their ideas for audio projects. You see a lot of this sort of thing in postings on the **Teensy Audio Forum**. Unfortunately, this approach leaves a lot to be desired in terms of a useful working device that can be conveniently carried around –without wires becoming disconnected – including standard input and output connectors, and other essential IO components, not to mention that the setup must be powered by a computer connected to Teensy over USB.

Before a desired final goal can be reached, however, such an experimental approach is essential to the process of design, testing, and verification. But while the format of the original Teensy Audio Tutorial and other small, simple, exploratory breadboard-based setups is sufficient for an initial evaluation, a more accommodating and capable platform is required for more advanced designs that must usable in real-world applications.

Help Wanted

Since I first realized and understood the potential of Teensy-based audio systems, I'd begun to think more and more about my idea of a simple yet powerful, carry-around audio device. The basic concept, described previously in the ***Audio Architecture*** section, was simple: sound sources (both internal and external) → effects → outputs. The design would also include a touchscreen, and several rotary encoders that could be used to select screens and options, and to enter settings for the various devices and components. Settings would be stored as *presets* so that a given setup could be recalled on demand.

But for this to become a reality, my PCB design and electronics skills would no longer be sufficient to create the sort of small and portable *physical* device I imagined—for now, I was back to the breadboard and jumper phase.

Help is On the Way

Fortunately, on the Teensy User Forum, I became aware of **ProtoSupplies.com**, a supplier of all manner of electronic components and devices, useful to both novices and professionals, for prototyping and testing their designs. And this discovery would become my ticket on the train from Audio Shield to AudioPlatform.

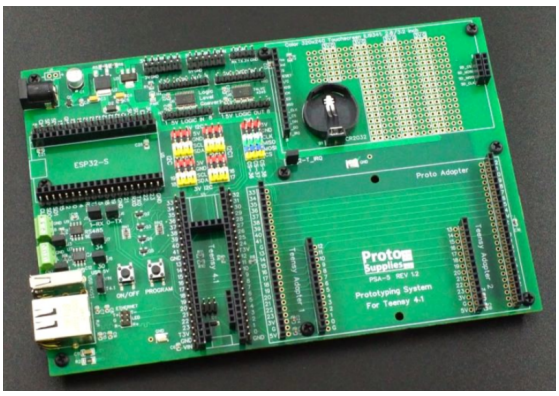
Ken Hahn, the proprietor of Protosupplies.com, is a regular visitor and contributor to the Teensy Forum, and has been instrumental to the design and development of AudioPlatform. His always helpful ideas, generous provision of components and modifications, and deep experience with electronic design have all been critical to moving the project steadily and successfully forward.

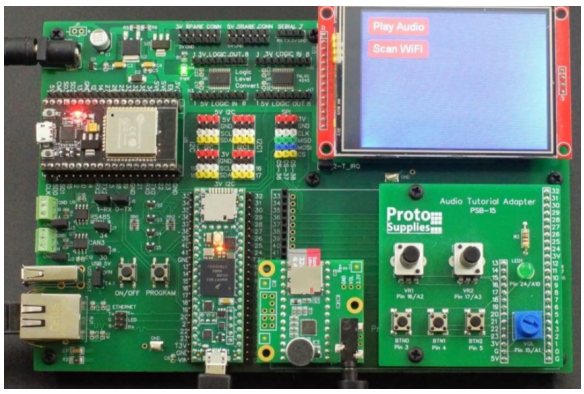
AudioPlatform in Stages

The process of moving beyond the Audio Shield / Breadboard setting to AudioPlatform was given essential support by from a pair of general purpose prototyping assemblies, designed and produced by ProtoSupplies.com. Employing the abundant resources of these assets, I was able to move forward smoothly, in advancing stages, to create a design that moved steadily closer to my goal.

Prototype No.1

The first AudioPlatform prototype employed ProtoSupplies' **Prototyping System for Teensy 4.1** depicted in the two images below. The baseboard (left image below) provides the core features required in many setups, including power input handling, logic level shifting, touch screen LCD for user interface and access to common buses and interfaces such as Ethernet, USB Host, CAN bus, RS485 bus, Serial TTL, I2C, level shifted I2C, SPI, level shifted SPI, WiFi and Bluetooth.





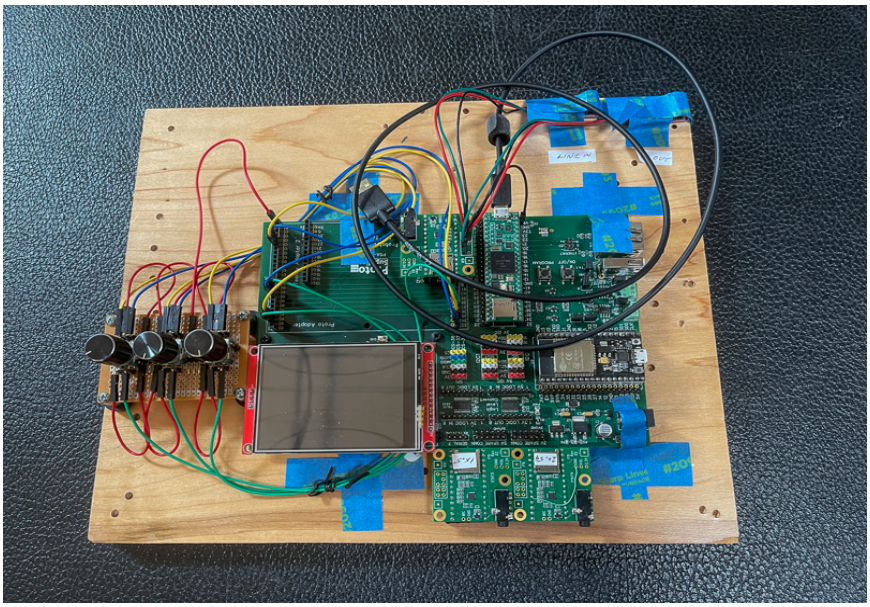
Prototyping System for Teensy 4.1

This system can be used as part of an elaborate breadboarding setup, with key parts of the system fully wired in, while also being able to work with temporary circuits built on solderless or soldered breadboards. The "fully-stuffed" version, (right image) which was used in the development of the first prototype, includes, among other features, several items central to the AudioPlatform design:

- Teensy 4.1 ***Hybrid Circuit Device***
- 16MB QSPI PSRAM
- 3.2″ TFT color 320×240 RGB SPI display w/ ILI9341 controller and touch w/ XPT2046 controller
- Teensy 4.x Rev D Audio Adapter, with a 16MB Serial Flash chip
- Teensy Audio Tutorial Adapter, featuring the additional components (buttons, faders) supplied in the original Tutorial Kit

The ESP32-S WiFi/Bluetooth coprocessor, connects to the Teensy 4.1 serial Rx2/Tx2 ports, is not used in the AudioPlatform 1.0 design, but will likely be offered as an option in a future version support of wireless IO of sensor or other control data

Seen below is an in-progress view of the first prototype version of AudioPlatform (note that the baseboard is rotated 180-degrees to make the LCD accessible at the bottom rather than the top of the device, and for more convenient access to other components).



AudioPlatform Prototype No. 1

From the beginning, the AudioPlatform sofware application has been designed for the use of 4 rotary encoders. But in this first version, only 3 were employed for testing, for the simple reason that only 3 would fit on the perf-board (lower right side), an old Radio Shack component on hand.

The mess of jumper wires connects the encoders to various pins of the Teensy 4.1, and the wound up black cable is a USB cable for connection of the Teensy to a PC for programming. The blue tape is used to hold the overall setup in place, and to secure two 3.5mm line input and output connectors in the upper right of the image.

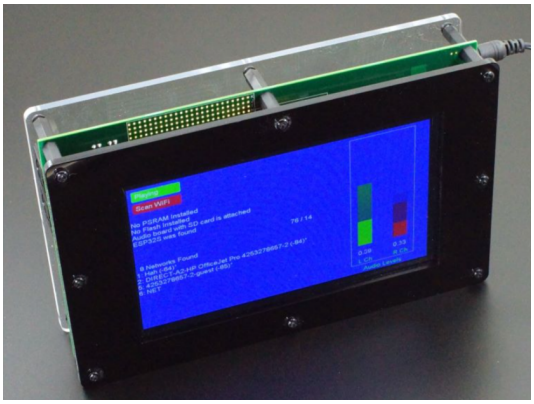
The overall assembly is mounted on a spare section of ½" wood for ease in moving about. However, the prototype design is pretty remote from the concept of a small, portable, and robust device that can be conveniently carried around.

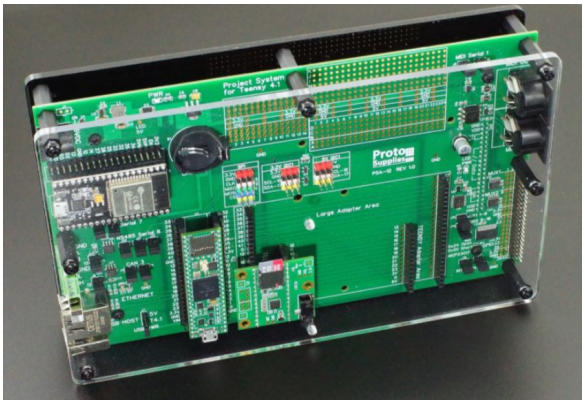
At the bottom of the image, right of center, are two additional Teensy 4.x Rev D Audio Adapters, each with a Serial Flash chip of different capacities, of different storage capacities, used for testing and determining the best size option. In the initial prototyping stage, SPI access to Serial Flash was possible using Flash chips soldered to the bottom of the Audio Adapter. In the final AudioPlatform version, the Flash chips, and all of the other components of the Audio Adapter would be located on the AudioPlatform's baseboard.

Prototype No. 2

As the software for the first prototype was beginning to come together, with user-interactive screens designed for the 320x240 ILI9341 display, Protosupplies.com introduced the **Project System For Teensy 4.1**.

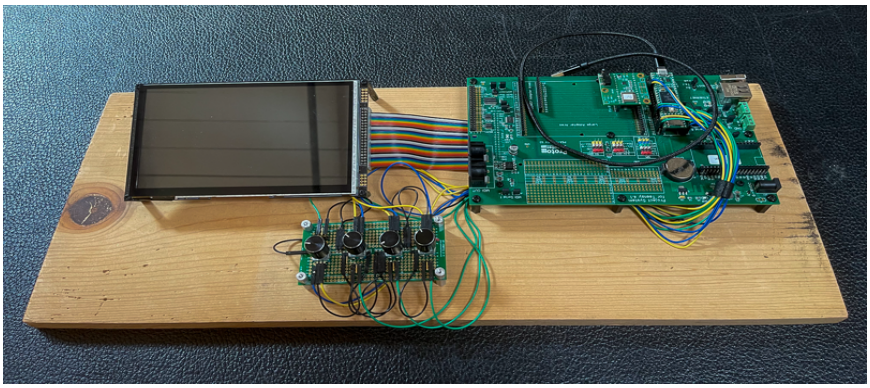
The new system included most of the same components present on the original Prototyping System, with a similar baseboard layout to that system, but also featured a number of variations. Of interest for the AudioPlatform was the addition of MIDI I/O ports using the original standard DIN5 MIDI connectors—most MIDI devices use USB I/O connections, but in recent years, a growing number of manufacturers are also including the DIN5 ports for support of legacy device. But the most important difference from the Project System the introduction of a 7-inch, 800x480 pixel RA8875 LCD touchscreen display.





Project System for Teensy 4.1

Seen below is an in-progress view of the second prototype version of AudioPlatform (note that the LCD display is removed from its position on the top panel and rotated to make both the LCD and components on the opposite side accessible during development).



AudioPlatform Prototype No. 2

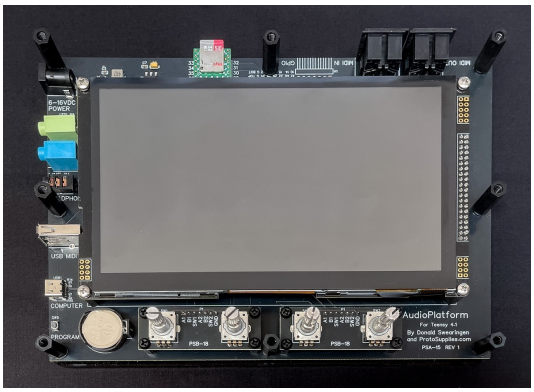
The second prototype also now included the 4 rotary encoders originally specified for AudioPlatform. And, with the addition of the larger touchscreen display, the hardware profile required by the AudioPlatform application was complete.

AudioPlatform 1.0

As *apApp* development proceeded, with more and more features and capabilities becoming a reality, it was time to begin thinking about addressing the final requirement for my design: portability. And, although the resources of the Project System were perfect for the application, it was obvious that the development environment, with its jumper wires, unused components, lack of IO, and dependence on a plug-in audio adapter would not be practical—an appropriate hardware design remained out of reach.

At this point, I decided to ask Ken Hahn about the possibility of creating a new hardware design, based on a single PCB, that would include the LCD, encoders, and all the circuit components and IO connectors that would be required in the application. I knew this was a big ask, and I really did not expect a positive response. But I was wrong about that—Ken readily agreed, and began working on a new hardware design that would fulfill the requirements.

I can't overstate the value of Ken's efforts and contributions, applying knowledge and skills far above my personal level. Over a surprisingly small number of revisions and PCB production rounds, what emerged was the system I'd first imagined: *AudioPlatform 1.0*.



AudioPlatform 1.0

Thanks to Ken's skill and generous assistance, the final version turned out to be far more attractive and powerful than I'd imagined when I first began to think about such a possibility. So now, there exists a device with software one can utilize immediately in a variety of settings where audio and sound are employed. And, also, a hardware platform designed to support audio systems prototyping and development of a wide range of potential user-defined applications.

Appendix

References

AudioPlatform Source and Documentation

AudioPlatform Source Code

AudioPlatform Documentation

ProtoSupplies.com

Web Site

Prototyping System For Teensy 4.1

Teensy 4.1 Fully Loaded For Prototyping System

Project System For Teensy 4.1

Teensy 4.x Audio Adapter

PJRC / Teensy

PJRC

Teensy User Forum

Teensy 4.1

Teensyduino

Teensyduino Libraries

Teensy Audio Shield

Teensy Audio Tutorial

Teensy DIY Audio Projects and Kits

Teensy Forum

TeensyMix Synth - DIY 8 Voice Synthesizer

Shruti II Teensy 4.1 DIY Synthesizer

Polyhonic Teensy DIY Sampler

Teensy Super Audio Board

MicroDexed Touch Synthesizer

Polyhonic Teensy DIY Sampler

YouTube

DIY Dirtywave M8 headless using a Teensy 4.1

DX7 Clone on Teensy 4.0

Electrotechnique TSynth - Teensy 4.1 based DIY Synthesizer

Teensy 4.1 Drone Synth

Teensy MIDI Step Sequencer

FALA Synthesizer - 8 Voice DIY Teensy Synth

Glossary

Hybrid Circuit Device

A hybrid circuit device, also known as a *hybrid integrated circuit* (HIC) or simply a hybrid circuit, is a miniaturized electronic circuit that combines discrete components (like transistors and diodes) and passive components (like resistors and capacitors) onto a single substrate. **https://en.wikipedia.org/wiki/Hybrid_integrated_circuit.**